

Project Overview: The Daily Crumb Web Application

The Daily Crumb is a front-end web application created for a fictional local bakery. The main purpose of the website is to give customers a simple and convenient way to browse available pastries, add their preferred items to a shopping cart, and place an order for pickup.

The application was built using HTML, CSS, and JavaScript. HTML provides the structure of each page, CSS controls the appearance and layout, and JavaScript handles the interactive parts of the website, such as displaying products, managing the shopping cart, calculating prices, and validating pickup times.

The project consists of three main pages and one JavaScript file:

- `index.html` – the home page
- `menu.html` – the product menu
- `checkout.html` – the shopping cart and checkout page
- `app.js` – the main application logic

Together, these files create a complete and connected shopping experience.

Home Page (`index.html`)

The home page acts as the bakery's digital storefront. Since it is the first page customers see, it was designed to feel welcoming and encourage them to explore the menu.

The page begins with a standard HTML5 structure. A viewport meta tag was included to make sure the website adjusts correctly across different screen sizes, including mobile phones, tablets, and desktop computers.

Two Google Fonts were used throughout the website. Inter was selected for paragraph text because it is clean and easy to read, while Playfair Display was used for headings to give the bakery a more elegant and traditional appearance. Together, these fonts help the website feel polished without making it look overly formal.

At the top of the page, there is a header containing the bakery's name, a small brand icon, and a navigation menu. The navigation links allow users to move easily between the Home, Menu, and Checkout pages.

The main section of the page is the hero area. It contains a short introduction to the bakery, an inviting headline, and a button that takes the customer directly to the menu.

Next to the text is an image slideshow. The HTML provides the basic slideshow container and the previous and next buttons, while JavaScript is responsible for inserting and changing the images.

Below the hero section, three featured bakery items are displayed in a grid. Each product is placed inside an `article` element and includes an image, a product name, and a short description. These items are written directly into the HTML because they are intended to remain as permanent featured products rather than being generated dynamically.

The page ends with a simple footer containing the bakery's copyright information.

Product Menu (`menu.html`)

The menu page contains the main shopping experience. This is where customers can view all available products and add them to their order.

The header, navigation bar, and footer use the same design as the home page. Keeping these elements consistent makes the website easier to navigate and gives all pages a unified appearance.

The most important element on this page is an empty `div` with the ID `menu-grid`.

The product cards are not written manually inside the HTML. Instead, JavaScript reads the bakery's product data and creates each card automatically when the page loads.

This approach makes the application easier to maintain. For example, when a new pastry is added, the product only needs to be added to the JavaScript product list. There is no need to write a new HTML card manually.

Each generated product card displays the item's image, name, price, and the appropriate cart controls. Customers can either add a product to their box or adjust its quantity using the plus and minus buttons.

Checkout Page (`checkout.html`)

The checkout page is the final step of the ordering process. It allows customers to review the products they selected, see the total price, choose a pickup time, and place their order.

The page uses a two-column layout to keep the information clear and organized.

On the left side, there is an empty container with the ID `cart-items`. JavaScript uses this area to display the products currently stored in the customer's cart.

Each item shows its name, price, quantity, and controls for updating or removing it. Since every customer may select different products, this section has to be generated dynamically.

The right side contains the order summary. It displays the total price of all products in the cart and includes the pickup form.

For the pickup time, the form uses an HTML input with the type set to `time`. This allows the browser to display its built-in time-selection interface, which is especially convenient on mobile devices.

The page also includes areas for instructions and error messages. These messages help guide the customer if something is missing or incorrect. For example, the application displays an error if the customer tries to place an order without selecting a pickup time or chooses a time that does not give the bakery enough preparation time.

Application Logic (`app.js`)

The `app.js` file controls the interactive behavior of the entire website. It connects the three pages, manages product information, remembers the customer's cart, performs calculations, and validates the final order.

Product Data

At the beginning of the file, an array called `menuItems` stores the available bakery products.

Each product is represented as an object containing information such as:

- A unique ID
- The product name
- The price
- The path to the product image

Keeping the product information in one array makes it easier to display and manage the same data across different pages.

Saving the Shopping Cart

Customers need to be able to move from the menu page to the checkout page without losing their selected items. To support this, the application uses the browser's local storage.

A constant called `STORAGE_KEY` stores the name `crumbCart`. This key is used whenever the application saves or retrieves the cart.

The `getCart` function checks whether a cart has already been saved in local storage. Since local storage saves information as text, `JSON.parse` is used to convert the saved text back into a JavaScript object.

The `saveCart` function performs the opposite operation. It uses `JSON.stringify` to convert the cart object into text before saving it in local storage.

This means the cart remains available even when the user moves between pages or refreshes the website.

Cart Management

Several functions were created to keep the shopping-cart logic organized.

The `addItemToCart` function checks whether a selected product is already in the cart. If it is, the function increases its quantity by one. If it is not, the product is added with a starting quantity of one.

The `removeItemFromCart` function removes a product completely from the cart.

The `updateCartQuantity` function allows the quantity of a specific product to be changed. If the new quantity is zero or less, the item is removed automatically. This prevents the cart from storing products with invalid quantities.

The `getTotalItems` function calculates the total number of pastries in the cart. This includes all quantities rather than simply counting the number of different product types.

The `getTotalPrice` function calculates the cost of the order by multiplying each product's price by its selected quantity and adding the results together.

Pickup-Time Calculation

The application also includes a practical rule for calculating how much preparation time the bakery needs.

The `calculatePrepMinutes` function begins with a base preparation time of 15 minutes. It then adds five additional minutes for every item in the cart.

For example, an order containing three items would require:

- 15 minutes of basic preparation time
- 15 additional minutes for the three items
- 30 minutes in total

The `getEarliestPickupTime` function adds the calculated preparation time to the current time. The result is used as the earliest time the customer is allowed to select.

This prevents customers from choosing an unrealistic pickup time and reflects how a real bakery would need time to prepare and pack an order.

Home-Page Slideshow

The `renderHeroSlideshow` function controls the slideshow on the home page.

It finds the slideshow container and creates image elements using the product images stored in `menuItems`. The slideshow automatically moves to the next image every four seconds.

Customers can also move through the images manually by using the previous and next buttons.

This gives the home page more movement and allows visitors to see several bakery products without taking up too much space.

Menu-Page Rendering

The `renderMenuPage` function runs when the customer opens the menu page.

It loops through the `menuItems` array and creates a product card for every available item. Each card includes the product image, name, price, and cart controls.

The function also checks local storage to see whether a product is already in the customer's cart.

When an item has not been selected, the card displays an **Add to Box** button. Once the product is added, the button is replaced with plus and minus controls so the customer can adjust the quantity.

The menu is updated each time the cart changes, ensuring that the displayed quantities always match the saved cart data.

Checkout-Page Rendering

The `renderCheckoutPage` function displays the customer's selected items on the checkout page.

It reads the cart from local storage, matches each cart entry with its product information, and creates the required HTML elements.

The function also calculates and displays the total price and determines the earliest valid pickup time.

When the customer submits the form, JavaScript prevents the page from refreshing immediately. It then checks several conditions:

1. The cart must not be empty.
2. A pickup time must be selected.
3. The selected time must not be earlier than the required preparation time.

When any of these checks fail, the customer receives a clear error message explaining what needs to be corrected.

When all the information is valid, the order is treated as successfully placed. The saved cart is cleared, and the application resets so that a new order can be created.

Event Delegation

The application uses event delegation to handle buttons efficiently.

Instead of adding a separate event listener to every product button, one listener is attached to the main product container. When the customer clicks inside the container, JavaScript checks which element was selected and what action it represents.

For example, clicking a plus button adds another item, while clicking a minus button reduces the quantity.

This method keeps the code shorter and makes it easier to manage buttons that are created dynamically.

Overall User Experience

The final application provides a clear and simple ordering journey.

Customers begin on the home page, where they are introduced to the bakery and shown a slideshow of available products. From there, they can open the menu and browse the full product selection.

When a customer adds an item, the cart is saved automatically in the browser. They can increase or decrease quantities without leaving the menu page.

At checkout, the customer can review the order, confirm the total cost, and select a realistic pickup time. The application checks the information before allowing the order to be submitted.

After a successful order, the cart is cleared and the website is ready for the next order.

Conclusion

The Daily Crumb project demonstrates the main skills required to build an interactive front-end web application.

It combines semantic HTML, responsive CSS, JavaScript-generated content, browser storage, form validation, event handling, and practical business rules in one connected project.

More importantly, the application was designed around a realistic customer journey. Every page has a clear purpose, from introducing the bakery to displaying products and completing an order. The result is a functional and user-friendly bakery website that could be expanded further in the future with features such as online payment, customer accounts, live inventory tracking, and a back-end ordering system.